

SOFTWARE REQUIREMENTS SPECIFICATION

Campus Library Resource Search Engine

Unified Search & Catalog System

Document ID	SRS-CLRSE-001
Version	1.0
Status	Draft
Date	April 2, 2026
Project Name	Campus Library Resource Search Engine
Prepared By	Software Engineering Team
Reviewed By	Project Supervisor / Faculty Advisor
Approved By	Pending Approval

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements for the Campus Library Resource Search Engine. It serves as the primary reference for all stakeholders including developers, project supervisors, testers, and administrators throughout the development lifecycle.

1.2 Project Overview

The campus library currently maintains a large collection of books, journals, and digital resources distributed across multiple publishers and platforms. Students are required to search these platforms separately, leading to a fragmented and inefficient research experience.

This project aims to build a unified search engine and resource catalog that aggregates all library holdings in one place, providing students with keyword search, metadata browsing, and direct links to digital resources.

1.3 Scope

The Campus Library Resource Search Engine will include the following components:

- A web-based search interface for students
- An administrative panel for managing the book/resource catalog
- A backend API service for data processing and business logic
- An Elasticsearch-powered search indexing system
- An event-driven synchronization pipeline using Kafka and the Outbox Pattern
- A SQL database for persistent storage of resource and catalog data

1.4 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
UI	User Interface
CRUD	Create, Read, Update, Delete
ES	Elasticsearch
Kafka	Apache Kafka — distributed event streaming platform

Outbox Pattern	Design pattern ensuring reliable event publication via a database outbox table
SQL	Structured Query Language
REST	Representational State Transfer
NFR	Non-Functional Requirement
FR	Functional Requirement
Admin	System Administrator

1.5 References

- Campus Software Engineering Project Brief — Section 4: Campus Library Resource Search Engine
- System Architecture Diagram — Campus Library Resource Search Engine (Eraser.io, April 2026)
- IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications
- Elasticsearch Reference Documentation — elastic.co/guide
- Apache Kafka Documentation — kafka.apache.org

2. Overall Description

2.1 Product Perspective

The Campus Library Resource Search Engine is a new, standalone web-based system designed to replace the current manual, multi-platform search process. It interfaces with a SQL database for storage, Elasticsearch for search indexing, and uses Kafka as a message broker to keep search indexes in sync with the catalog database.

The system is composed of four major layers as illustrated in the architecture diagram:

- Client Layer: Search UI (student-facing) and Admin Panel (admin-facing)
- Backend Service Layer: RESTful API + Book Service + Outbox Publisher
- Message Broker Layer: Apache Kafka for event-driven communication
- Data Layer: SQL Database (Books Table + Outbox Table) and Search Layer (Elasticsearch + Kafka Consumer)

2.2 Product Functions

At a high level, the system provides the following primary functions:

- Unified catalog search with keyword, partial, and synonym matching
- Filtering of results by author, title, subject, and publication year
- Metadata display including title, authors, publisher, and external resource links
- Admin-managed CRUD operations on the library catalog
- Real-time index synchronization from the catalog database to Elasticsearch via Kafka

2.3 User Classes and Characteristics

User Class	Description	Access Level
Student / Researcher	Primary end-users searching for library resources	Read-only access to search UI and resource metadata
Library Administrator	Staff responsible for maintaining the catalog	Full CRUD access via admin panel
System Developer	Responsible for deployment and maintenance	Full system and infrastructure access

2.4 Operating Environment

- Web browser-based front-end (Chrome, Firefox, Safari, Edge — latest 2 versions)
- Backend deployable on Linux-based server (Ubuntu 22.04 LTS or later)
- Elasticsearch 8.x for search indexing
- Apache Kafka 3.x for message brokering

- PostgreSQL or MySQL as the SQL database backend

2.5 Design and Implementation Constraints

- Must use Elasticsearch as the primary search engine (Apache Solr as alternative)
- Event synchronization between the database and search index must implement the Outbox Pattern to guarantee message delivery
- The API must be RESTful
- The system must be accessible via web browser; no native mobile app is in scope
- All catalog data modifications must pass through the Admin Panel and API — no direct database access

2.6 Assumptions and Dependencies

- Library catalog data (books, journals, digital resources) is available for import or manual entry
- Students and admins have access to a modern web browser and campus network
- Elasticsearch and Kafka infrastructure will be provisioned as part of deployment
- External publisher websites and digital resource URLs are valid and maintained by third parties

3. Functional Requirements

3.1 Student Search Interface (Search UI)

The Search UI is the primary student-facing component. It must allow students to find library resources efficiently.

3.1.1 Search Input and Execution

ID	Priority	Requirement	Source
FR-S01	High	The system shall allow users to enter free-text keywords in a search bar and submit a search query.	Project Brief §4.2, §4.3
FR-S02	High	The system shall support partial keyword matching (prefix/infix search).	Project Brief §4.3
FR-S03	High	The system shall rank search results by relevance using Elasticsearch scoring.	Project Brief §4.3
FR-S04	Medium	The system shall support basic synonym matching to broaden search recall.	Project Brief §4.3
FR-S05	Medium	The system shall return results within 2 seconds for typical queries under normal load.	NFR Derivation

3.1.2 Filtering and Refinement

ID	Priority	Requirement	Source
FR-S06	High	The system shall allow users to filter search results by author name.	Project Brief §4.2
FR-S07	High	The system shall allow users to filter results by resource title.	Project Brief §4.2
FR-S08	High	The system shall allow users to filter results by subject or topic.	Project Brief §4.2
FR-S09	High	The system shall allow users to filter results by publication year or year range.	Project Brief §4.2
FR-S10	Medium	Filters shall be combinable (e.g., author + year range simultaneously).	NFR Derivation

3.1.3 Result Display and Metadata

ID	Priority	Requirement	Source
FR-S11	High	Each search result shall display the resource title, author(s), publisher, and publication year.	Project Brief §4.2

FR-S12	High	Each result shall include a clickable link that redirects the user to the publisher website or digital resource.	Project Brief §4.2
FR-S13	Medium	The system shall display a paginated list of results (default: 10 results per page).	NFR Derivation
FR-S14	Low	Users shall be able to view an expanded metadata view for a selected resource.	NFR Derivation

3.2 Admin Panel

The Admin Panel enables library administrators to manage the catalog through CRUD operations.

ID	Priority	Requirement	Source
FR-A01	High	Administrators shall be able to add new book/resource records to the catalog via the admin panel.	Architecture Diagram, Project Brief §4.5
FR-A02	High	Administrators shall be able to edit existing resource records (title, author, publisher, subject, year, URL).	Architecture Diagram
FR-A03	High	Administrators shall be able to delete resource records from the catalog.	Architecture Diagram
FR-A04	High	All admin operations shall require authentication before access is granted.	Security NFR
FR-A05	Medium	The admin panel shall display a list of all catalog entries with search and filter capability.	Usability NFR
FR-A06	Medium	Admin actions shall be logged with a timestamp and user identity for audit purposes.	NFR Derivation

3.3 Backend API Service

The API layer handles all business logic and acts as the gateway between the frontend clients and backend services.

ID	Priority	Requirement	Source
FR-B01	High	The system shall expose a RESTful API for all CRUD operations on catalog resources.	Architecture Diagram
FR-B02	High	The API shall validate all incoming request data and return appropriate HTTP error codes on failure.	NFR Derivation

FR-B03	High	On a successful catalog write (add/edit/delete), the Book Service shall persist the change to the Books Table in the SQL database.	Architecture Diagram
FR-B04	High	On a successful catalog write, the Book Service shall write a corresponding event record to the Outbox Table in the same database transaction.	Architecture Diagram — Outbox Pattern
FR-B05	High	The API shall expose a search endpoint that proxies queries to Elasticsearch and returns ranked results.	Architecture Diagram

3.4 Outbox Publisher & Kafka Integration

This component ensures reliable event delivery from the database to the Kafka message broker.

ID	Priority	Requirement	Source
FR-O01	High	The Outbox Publisher shall poll the Outbox Table at regular intervals to detect new unpublished events.	Architecture Diagram
FR-O02	High	For each new event, the Outbox Publisher shall publish the event to the appropriate Kafka topic.	Architecture Diagram
FR-O03	High	After successful Kafka acknowledgment, the Outbox Publisher shall mark the event as published in the Outbox Table.	Architecture Diagram
FR-O04	Medium	The system shall retry failed Kafka publish attempts with an exponential backoff strategy.	Reliability NFR

3.5 Search Layer (Kafka Consumer & Elasticsearch)

ID	Priority	Requirement	Source
FR-E01	High	The Kafka Consumer shall subscribe to catalog event topics and consume new events in order.	Architecture Diagram
FR-E02	High	Upon receiving an 'add' or 'update' event, the Kafka Consumer shall index or update the corresponding document in Elasticsearch.	Architecture Diagram
FR-E03	High	Upon receiving a 'delete' event, the Kafka Consumer shall remove the corresponding document from the Elasticsearch index.	Architecture Diagram
FR-E04	Medium	The Elasticsearch index shall be kept eventually consistent with the Books Table via this pipeline.	Architecture Diagram

4. Non-Functional Requirements

4.1 Performance

ID	Priority	Requirement	Source
NFR-P01	High	Search queries shall return results within 2 seconds under normal load (up to 100 concurrent users).	Project Brief
NFR-P02	High	The system shall support at least 100 simultaneous users without degradation.	Scalability
NFR-P03	Medium	Catalog updates shall be reflected in search results within 10 seconds (eventual consistency).	Architecture Diagram

4.2 Reliability & Availability

ID	Priority	Requirement	Source
NFR-R01	High	The system shall achieve 99.5% uptime during campus operating hours.	Project Brief
NFR-R02	High	The Outbox Pattern shall guarantee at-least-once event delivery to Kafka — no catalog updates shall be permanently lost.	Architecture Diagram
NFR-R03	Medium	The system shall gracefully handle Elasticsearch downtime by queuing events until the service recovers.	Reliability

4.3 Security

ID	Priority	Requirement	Source
NFR-S01	High	Admin panel access shall require username/password authentication.	Security
NFR-S02	High	All API endpoints that modify data shall require an authenticated session token.	Security
NFR-S03	High	The system shall protect against SQL injection and XSS attacks.	Security
NFR-S04	Medium	User passwords shall be stored using a strong hashing algorithm (bcrypt or Argon2).	Security
NFR-S05	Medium	HTTPS shall be enforced for all client-server communication.	Security

4.4 Usability

ID	Priority	Requirement	Source
NFR-U01	High	The search interface shall be accessible via standard web browsers without requiring plugins or installation.	Project Brief §4.5
NFR-U02	High	The UI shall display results in a clear, readable layout with distinguishable metadata fields.	Project Brief
NFR-U03	Medium	Error messages shall be user-friendly and actionable.	Usability
NFR-U04	Medium	The admin panel shall provide form validation feedback in real time.	Usability

4.5 Maintainability

ID	Priority	Requirement	Source
NFR-M01	Medium	The codebase shall follow consistent coding standards documented in a developer guide.	Maintainability
NFR-M02	Medium	Each system component (API, Consumer, Publisher) shall be independently deployable.	Architecture
NFR-M03	Low	System logs shall be structured (JSON format) and retained for at least 30 days.	Operations

5. System Architecture Overview

The following describes the four-layer architecture of the Campus Library Resource Search Engine as depicted in the architecture diagram.

Component	Technology	Responsibility
Search UI	React / HTML+CSS+JS	Student-facing web application for submitting queries and viewing results
Admin Panel (Book Management UI)	React / HTML+CSS+JS	Administrator interface for catalog CRUD operations
API Gateway	Node.js / Express or similar REST framework	Receives all requests from both UI clients; routes to Book Service
Book Service	Backend business logic module	Handles catalog logic; writes to Books Table and Outbox Table atomically
Outbox Publisher	Background polling service	Polls Outbox Table and publishes pending events to Kafka
Apache Kafka	Apache Kafka 3.x	Message broker that decouples catalog writes from search index updates
Kafka Consumer	Event consumer service	Consumes Kafka events and updates Elasticsearch index accordingly
Elasticsearch	Elasticsearch 8.x	Full-text search engine; stores indexed book documents for fast retrieval
SQL Database	PostgreSQL / MySQL	Persistent storage for Books Table and Outbox Table

5.1 Data Flow — Catalog Write Path

1. Admin submits a CRUD operation via the Book Management UI.
2. The Admin Panel sends a request to the API.
3. The API delegates to the Book Service.
4. The Book Service atomically writes the book record to the Books Table AND an event record to the Outbox Table in a single database transaction.
5. The Outbox Publisher polls the Outbox Table, detects the new event, and publishes it to Kafka.
6. The Kafka Consumer reads the event and updates the Elasticsearch index (add/update/delete).

5.2 Data Flow — Search Read Path

7. A student enters a search query in the Search UI.
8. The Search UI sends the query to the API search endpoint.
9. The API forwards the query to Elasticsearch.
10. Elasticsearch returns ranked results based on indexed book data.
11. The API returns the results to the Search UI for display.

6. Project Deliverables

The following deliverables are required per the project brief (Section 4.5):

Deliverable	Description	Status
Web-based Search Interface	Student-facing search UI (React or equivalent)	To Be Developed
Resource Catalog Database	SQL database with Books and Outbox tables	To Be Developed
Search Indexing System	Elasticsearch index + Kafka Consumer pipeline	To Be Developed
Administrative Management Panel	Admin CRUD UI and authenticated backend	To Be Developed
Backend API	RESTful API service (Book Service, Outbox Publisher)	To Be Developed
System Architecture Document	Architecture diagram and data flow documentation	Complete
SRS Document	This document	Draft

7. Appendix

7.1 Revision History

Version	Date	Description	Author
1.0	April 2, 2026	Initial draft based on project brief and architecture diagram	SWE Team

7.2 Open Issues

- Authentication mechanism (JWT vs session-based) to be confirmed with project supervisor.
- Database choice (PostgreSQL vs MySQL) to be finalized during technical design phase.
- Kafka topic naming conventions and partition strategy to be documented in a separate design document.
- UI framework (React vs Vue vs plain HTML) to be decided by the development team.
- Synonym dictionary / thesaurus source for Elasticsearch synonym matching to be identified.

7.3 Out of Scope

- Mobile native applications (iOS / Android)
- User accounts or personalization features for students
- Integration with external library systems (e.g., OPAC, WorldCat)
- E-book/journal full-text search (only metadata is indexed)
- Automated catalog data import pipelines from publisher feeds